

PanoramaScore Implementation Report (Phase 1)

Date: May 23, 2025

Submitted By: Asad Nawaz

Objective: Provide technical implementation details of the foundational components built for the PanoramaScore system, including document ingestion, metadata analysis, and field extraction with a working UI prototype.

Overview

The current implementation focuses on building the core foundational agents and interface that allow users to upload documents, analyze metadata, extract structured information, and display results. This forms the groundwork for domain-specific risk scoring in future stages.

Implemented Components

1.Document Ingest Agent

Purpose: Accept PDF, image, or CSV files from the user and save them to the system.

Implementation Details:

- * Built using FastAPI with a '/upload' endpoint
- * Accepts file types: '.pdf', '.jpg', '.png', '.jpeg', '.csv'
- * Uploaded files are saved to a temporary folder with a unique UUID
- * Validates file types and sizes
- * Generates real-time upload feedback to the user

2. Metadata Hashing Agent

Purpose: Extract basic metadata from the uploaded document for identification, traceability, and versioning.

Captured Metadata:

- * 'filename'
- * 'upload_time' (UTC Timestamp)
- * 'file_size' (both readable and in bytes)
- * 'checksum' using SHA-256 (via Python's 'hashlib')
- * 'client_id' (optional, if passed)

Output Format:

```
"""json
{
  "filename": "doc.pdf",
  "upload_time": "2025-05-22T17:00:00Z",
  "file_size": "245KB",
  "file_size_bytes": 250119,
  "checksum": "9ab6...f10d"
}
"""
```

3. OCR & Text Normalization Agent

Purpose: Extract readable text from uploaded documents and clean the output.

Implementation Details:

- * Used 'pdfplumber' for text-based PDFs and fallback OCR logic for scanned files (Tesseract-ready)
- * Stripped unwanted characters, line breaks, and headers
- * Normalized text for clean downstream processing

Output:

- * Full plain text in UTF-8 format
- * Stored temporarily for field extraction use

4. Field Extraction (NLU Agent)

Purpose: Detect and extract key fields from document text for risk scoring

Fields Extracted:

- * 'amount'
- * 'date'
- * 'document_type'
- * (Future extension for 'name', 'claim_number', 'location', etc.)

****Field Extraction Strategy:****

- * Rule-based extraction using 'regex'
- * Assigns 'confidence scores' to each field (scale: 0.0–1.0)

Output Format:

```
““json
{
  "extracted_text": "...",
  "fields": {
    "amount": "$5000.00",
    "date": "2025-05-22",
    "document_type": "Insurance Claim"
  },
  "confidence_scores": {
    "amount": 0.7,
    "date": 0.6,
    "document_type": 0.9,
    "name": 0.0
  }
}
““
```

UI Demo (FastAPI + Swagger)

A minimal and responsive UI has been developed using FastAPI's Swagger UI that:

- * Allows document upload directly from browser
- * Displays metadata and extraction results in real-time
- * Supports demo viewing and output interpretation for end users

Next Steps

- * Integrate vector storage (pgvector) and embeddings for document similarity (RAG)
- * Add YAML domain adapters and risk scoring logic
- * Expand field extraction with NLP + ML-based enhancements